
Formalizing Group Action in Lean

Student name: *Frankie Feng-Cheng WANG*

Email: *maths@frankie.wang*

GitHub: *<https://github.com/FrankieeW/GroupAction>*

Course: *MATH70040-Formalising Mathematics* – Lecturer: *Dr Bhavik Mehta*

Due date: *January 29, 2026*

Contents

1	Introduction	2
1.1	Main result	2
1.2	Why formalise group actions?	2
2	Background and Design Choices	2
2.1	Why a custom action class	2
2.2	What Lean is checking	2
3	Definitions	2
3.1	Lean code	3
4	Concrete Examples of Group Actions	3
4.1	Symmetric group acting on a set	3
4.2	Group acting on itself by left multiplication	4
4.3	Subgroup acting on the group	4
4.4	Conjugation action of a subgroup on itself	4
4.5	Scalar action on complex vector spaces	5
4.6	Scalar action via units	5
4.7	Dihedral group action on a square	6
5	Permutation Representation	6
5.1	Proof sketch (informal)	6
5.2	Lean code	6
6	Stabilizers	8
6.1	Proof sketch (informal)	8
6.2	Lean code	8
7	What Lean Guarantees	9
8	Reflection	10
8.1	Specific challenges encountered	10
8.2	Potential extensions	10

9 AI Assistance Statement	10
10 References	11

1. Introduction

This report formalises the theory of group actions in Lean 4. I define a custom action class, construct the permutation representation, and prove that stabilizers form subgroups. The development is split across modules under `lean/GroupAction/` and collected by `lean/GroupAction.lean`.

1.1. Main result.

Theorem. Let X be a G -set. For each $g \in G$, the map $\varphi_g : X \rightarrow X$ defined by $\varphi_g(x) = g \cdot x$ is a permutation of X . The map $\Phi : G \rightarrow \text{Sym}(X)$ given by $\Phi(g) = \varphi_g$ is a group homomorphism, and for all $g \in G$ and $x \in X$, $\Phi(g)(x) = g \cdot x$.

1.2. Why formalise group actions?. Formalisation in Lean forces every step to be explicit: hidden assumptions become visible, informal “clearly” arguments must be justified, and type checking prevents mistakes. An external examiner familiar with group theory can read this report without prior Lean knowledge, as each section presents the mathematics first, then shows the corresponding Lean code. Lean’s type system ensures that the permutation representation really is a group homomorphism and that stabilizers really are subgroups—claims that informal proofs sometimes take for granted.

2. Background and Design Choices

2.1. Why a custom action class. Mathlib already provides `MulAction`, but I define a small custom class `GroupAction` with just two axioms: compatibility with multiplication and the identity action. This keeps the development minimal and makes it easy to map Lean lines to textbook lines. The cost is that I cannot use existing `MulAction` lemmas, but the benefit is transparency.

2.2. What Lean is checking. Lean treats each statement as a typed object. When I define `sigmaPerm`, Lean checks that the inverse function is a two-sided inverse. When I define `phi`, Lean checks that its image lies in the group `Equiv.Perm X` and that multiplication is permutation composition. This explicit checking prevents hidden gaps that can slip into informal proofs.

3. Definitions

This section introduces the group action class and the standing assumptions used throughout the file.

3.1. Lean code. I first declare a minimal class for group actions:

```

20 class GroupAction (G : Type*) [Monoid G] (X : Type*) where
21   -- The action map: apply g to x.
22   act : G → X → X
23   -- Compatibility with multiplication in G.
24   ga_mul : ∀ g₁ g₂ x, act (g₁ * g₂) x = act g₁ (act g₂ x)
25   -- The identity element acts as the identity function.
26   ga_one : ∀ x, act 1 x = x

```

Note that I use `Monoid` here because the definition of a group action does not require inverses. Later results about permutation representations and stabilizers require a `Group`, so the subsequent sections assume `[Group G]`.

Throughout the file I work with a group G acting on a type X :

```

13   -- Fix a group and an action instance.
14   variable {G : Type*} [Group G] {X : Type*} [GroupAction G X]

```

4. Concrete Examples of Group Actions

This section presents several concrete instances of the `GroupAction` class, illustrating how the abstract definition applies to familiar mathematical structures.

4.1. Symmetric group acting on a set. Let X be any type. The symmetric group $\text{Sym}(X)$ acts on X by evaluation: for $\sigma \in \text{Sym}(X)$ and $x \in X$, define $\sigma \cdot x := \sigma(x)$.

```

19 instance permGroupAction (X : Type*) : GroupAction (Equiv.Perm X) X :=
20   { act := fun g x => g x
21     ga_mul := by
22       intro g₁ g₂ x
23       rfl
24     ga_one := by
25       intro x
26       rfl }

```

The axiom proofs are immediate by reflexivity: composition of permutations and function evaluation commute definitionally in Lean.

Two standard properties of this action are faithful and transitive:

```

28 theorem permGroupAction_faithful (X : Type*) :
29   GroupAction.faithful (G := Equiv.Perm X) (X := X) :=
30   by
31     intro g₁ g₂ h
32     ext x
33     exact h x
34
35 theorem permGroupAction_transitive
36   (X : Type*) [DecidableEq X] :
37   GroupAction.transitive (G := Equiv.Perm X) (X := X) :=
38   by

```

```

39   intro x1 x2
40   refine <Equiv.swap x1 x2, ?_>
41   simp [GroupAction.act]

```

The instance `DecidableEq X` is required because `Equiv.swap` constructs a permutation by branching on whether two elements are equal, which needs decidable equality.

4.2. Group acting on itself by left multiplication. Every group G acts on itself by left multiplication: $g_1 \cdot g_2 := g_1 * g_2$.

```

44   instance groupAsGSet (G : Type*) [Group G] : GroupAction G G :=
45   { act := fun g1 g2 => g1 * g2
46     ga_mul := by
47       intro g1 g2 g3
48       rw [mul_assoc]
49     ga_one := by
50       intro g
51       rw [one_mul] }

```

The `ga_mul` axiom follows from associativity of group multiplication, and `ga_one` from the identity axiom.

4.3. Subgroup acting on the group. A subgroup $H \leq G$ acts on G by left multiplication. Elements of H are coerced to elements of G using $(h : G)$.

```

55   instance subgroupAsGSet {G : Type*} [Group G] (H : Subgroup G) :
56   ↪ GroupAction H G :=
57   { act := fun h g => (h : G) * g
58     ga_mul := by
59       intros
60       simp [mul_assoc]
61     ga_one := by
62       intros
63       simp }

```

The `simp` tactic handles the coercion and applies associativity and identity axioms.

4.4. Conjugation action of a subgroup on itself. A subgroup H acts on itself by conjugation: $h_1 \cdot h_2 := h_1 h_2 h_1^{-1}$.

```

66   instance subgroupAsGSetConjugation {G : Type*} [Group G] (H : Subgroup
67   ↪ G) : GroupAction H H :=
68   {
69     act := fun h g => h * g * h-1
70     ga_mul := by
71       intro g1 g2 g3
72       simp
73       -- goal state: g1 * g2 * g3 * (g2-1 * g1-1) = g1 * (g2 * g3 * g2-1) *
74       ↪ g1-1
75       rw [← mul_assoc g1]

```

```

74  -- goal state:  $g_1 * g_2 * g_3 * (g_2^{-1} * g_1^{-1}) = g_1 * (g_2 * g_3) * g_2^{-1} * g_1^{-1}$ 
    ↪  $g_1^{-1}$ 
75  rw [mul_assoc g1 g2]
76  -- goal state:  $g_1 * (g_2 * g_3) * (g_2^{-1} * g_1^{-1}) = g_1 * (g_2 * g_3) * g_2^{-1} * g_1^{-1}$ 
    ↪  $g_1^{-1}$ 
77  rw [← mul_assoc]
78  ga_one := by
79  intros
80  simp }

```

The `mul_assoc` lemma is from `mathlib` and states associativity of multiplication: `mul_assoc.{u1} {G : Type u1} [Semigroup G] (a b c : G) : a * b * c = a * (b * c)`. The `ga_mul` proof can easily be done by `simp [mul_assoc]`, but I expanded it to show the steps explicitly by using `rw`.

For example the `rw [← mul_assoc g1]` will automatically find the passible situation looks like $g_1 * (b * c)$, because we only give one argument g_1 to `← mul_assoc`. In detail, it will rewrite the left hand side $g_1 * (g_2 * g_3 * g_2^{-1})$ to $g_1 * (g_2 * g_3) * g_2^{-1}$ according to the right to left direction of the lemma `mul_assoc`.

4.5. Scalar action on complex vector spaces. The multiplicative group $\mathbb{C}^\times$ of nonzero complex numbers acts on $\mathbb{C}^n$ by scalar multiplication.

```

88  instance vectorSpaceAsCStarSet (n : ℕ) : GroupAction (ℂ×) (Fin n → ℂ) :=
89  { act := fun r v => fun i => (r : ℂ) * v i
90    ga_mul := by
91    intros r1 r2 v
92    ext i
93    simp [mul_assoc]
94    ga_one := by
95    intro v
96    ext i
97    simp }

```

Here $\mathbb{C}^\times$ denotes the units of $\mathbb{C}$ (invertible elements), and `Fin n → ℂ` represents functions from $\{0, \dots, n-1\}$ to $\mathbb{C}$ (i.e., n -dimensional complex vectors). The `ext` tactic proves function equality pointwise.

4.6. Scalar action via units. The unit group $\alpha^\times$ of a monoid α acts on α^n by scalar multiplication.

```

103  instance vectorSpaceAsUnitSet
104  (α : Type*) [Monoid α]
105  (n : ℕ) :
106  GroupAction (α×) (Fin n → α) :=
107  { act := fun r v => fun i => (r : α) * v i
108    ga_mul := by
109    intros r1 r2 v
110    ext i
111    simp [mul_assoc]
112    ga_one := by

```

```

113   intro v
114   ext i
115   simp }

```

This generalizes the complex example by abstracting over the coefficient monoid.

4.7. Dihedral group action on a square. Let D_4 be the symmetry group of a square, acting on $\mathbb{Z}/4\mathbb{Z}$.

```

134   abbrev D4 := DihedralGroup 4
135   abbrev Vertex := ZMod 4
136
137   def d4Act (g : D4) (x : ZMod 4) : ZMod 4 :=
138   match g with
139   | DihedralGroup.r i => x - i
140   | DihedralGroup.sr i => i - x
141
142   instance d4ActionZMod4 : GroupAction D4 (ZMod 4) :=
143   { act := d4Act
144     ga_mul := by
145       intro g1 g2 x
146       cases g1 <|> cases g2 <|>
147       simp [d4Act, sub_eq_add_neg] <|> ring
148     ga_one := by
149       intro x
150       simp [DihedralGroup.one_def, d4Act, -DihedralGroup.r_zero]
151   }

```

5. Permutation Representation

The core construction is the map $\sigma_g : X \rightarrow X$ given by $\sigma_g(x) = g \cdot x$. The inverse of σ_g is $\sigma_{g^{-1}}$, so σ_g is a permutation. This yields the permutation representation $\phi : G \rightarrow \text{Sym}(X)$.

5.1. Proof sketch (informal). To show that σ_g is a bijection, compute $\sigma_{g^{-1}}(\sigma_g(x)) = (g^{-1}g) \cdot x = 1 \cdot x = x$, and similarly $\sigma_g(\sigma_{g^{-1}}(x)) = x$. The representation is defined by $\phi(g) = \sigma_g$, and multiplicativity follows from the action axiom:

$$\phi(g_1 g_2)(x) = (g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = (\phi(g_1) \phi(g_2))(x).$$

5.2. Lean code. First, define the underlying action map:

```

26   /-- The action map `sigma g : X -> X`
27   given by `x ↦ g · x`. -/
28   def sigma (g : G) : X -> X :=
29   -- We use the action directly.
30   fun x => GroupAction.act g x

```

Then package it as a permutation using the inverse action:

```

33  /-- The permutation of `X` induced by `g`.
34  The inverse is given by the action of `g⁻¹`. -/
35  def sigmaPerm (g : G) : Equiv.Perm X :=
36  { toFun := sigma g
37    invFun := sigma g⁻¹
38    left_inv := by
39      intro x
40      -- Step 1: combine `g⁻¹` and `g` using the action axiom.
41      calc
42        GroupAction.act g⁻¹ (GroupAction.act g x) =
43        GroupAction.act (g⁻¹ * g) x := by
44          simp using (GroupAction.ga_mul g⁻¹ g x).symm
45      -- Step 2: simplify `g⁻¹ * g` to `1`.
46      _ = GroupAction.act (1 : G) x := by
47        simp
48      -- Step 3: the identity acts as the identity on `X`.
49      _ = x := GroupAction.ga_one x
50    right_inv := by
51      intro x
52      -- Step 1: combine `g` and `g⁻¹` using the action axiom.
53      calc
54        GroupAction.act g (GroupAction.act g⁻¹ x) =
55        GroupAction.act (g * g⁻¹) x := by
56          simp using (GroupAction.ga_mul g g⁻¹ x).symm
57      -- Step 2: simplify `g * g⁻¹` to `1`.
58      _ = GroupAction.act (1 : G) x := by
59        simp
60      -- Step 3: the identity acts as the identity on `X`.
61      _ = x := GroupAction.ga_one x }

```

Using `#check` on `sigmaPerm` confirms it has type

```

64  #check sigmaPerm

```

This confirms the map $g \mapsto \sigma_g$ is a well-defined function from G to $\text{Sym}(X)$. Define the representation and its basic properties:

```

66  /-- The permutation representation `phi : G → Equiv.Perm X`
67  induced by the action. -/
68  def phi (g : G) : Equiv.Perm X :=
69  sigmaPerm g
70
71  -- `phi` agrees with the action on elements.
72  lemma phi_apply (g : G) (x : X) : phi g x = GroupAction.act g x := rfl
73
74  lemma phi_mul (g₁ g₂ : G) :
75  phi (X := X) (g₁ * g₂) = phi (X := X) g₁ * phi (X := X) g₂ := by
76  -- Reduce equality of permutations to pointwise equality.
77  apply Equiv.ext
78  intro (x : X)
79  calc
80    phi (g₁ * g₂) x = GroupAction.act (g₁ * g₂) x := rfl
81    _ = GroupAction.act g₁ (GroupAction.act g₂ x) := GroupAction.ga_mul g₁
82      ↪ g₂ x

```

```

83 lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
84   apply Equiv.ext
85   intro x
86   calc
87     phi (1 : G) x = GroupAction.act (1 : G) x := rfl
88     _ = x := GroupAction.ga_one x
89     _ = (1 : Equiv.Perm X) x := by simp [Equiv.Perm.one_apply]

```

Finally, package the existence statement:

```

101 -- Existence of the permutation representation with the required
102   ↪ properties.
102 theorem group_action_to_perm_representation :
103   ∃ (ψ : G → Equiv.Perm X),
104   (∀ g x, ψ g x = GroupAction.act g x) ∧
105   (∀ g₁ g₂, ψ (g₁ * g₂) = ψ g₁ * ψ g₂) ∧
106   (ψ 1 = 1) := by
107   exact ⟨phi, ⟨phi_apply, ⟨phi_mul, phi_one⟩⟩⟩

```

6. Stabilizers

For a fixed $x \in X$, the stabilizer is the subset

$$G_x = \{g \in G \mid g \cdot x = x\}.$$

In Lean this is first defined as a set, then shown to be the carrier of a subgroup, and finally packaged as a Subgroup.

6.1. Proof sketch (informal). The identity element fixes every x , so $1 \in G_x$. If g_1 and g_2 fix x , then $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = g_1 \cdot x = x$, so $g_1 g_2 \in G_x$. If g fixes x , then $g^{-1} \cdot x = g^{-1} \cdot (g \cdot x) = (g^{-1} g) \cdot x = x$, so $g^{-1} \in G_x$.

6.2. Lean code. Start from the set definition:

```

22 /-- The stabilizer set
23   `G_x = { g ∈ G | g · x = x }`. -/
24 def stabilizerSet (x : X) : Set G :=
25   -- Membership means g fixes x.
26   { g : G | GroupAction.act g x = x }

```

Package it as a subgroup by checking closure:

```

27 /-- The stabilizer `G_x` as a subgroup of `G`. -/
28 def stabilizer (x : X) : Subgroup G := by
29   exact
30     { carrier := stabilizerSet (G := G) (X := X) x
31       one_mem' := by
32         -- The identity fixes every x.
33         simp [stabilizerSet, GroupAction.ga_one x]
34       mul_mem' := by
35         intro g₁ g₂ hg₁ hg₂
36         calc

```



```

37   -- Closure under multiplication uses the action axiom.
38   GroupAction.act (g1 * g2) x = GroupAction.act g1 (GroupAction.act g2
   ↪ x) := by
39   simp using (GroupAction.ga_mul g1 g2 x)
40   _ = GroupAction.act g1 x := by
41   rw [hg2]
42   _ = x := hg1
43   inv_mem' := by
44   intro g hg
45   calc
46   -- If g fixes x then g-1 fixes x.
47   GroupAction.act g-1 x = GroupAction.act g-1 (GroupAction.act g x) :=
   ↪ by
48   rw [hg]
49   _ = GroupAction.act (g-1 * g) x := by
50   simp using (GroupAction.ga_mul g-1 g x).symm
51   _ = GroupAction.act (1 : G) x := by
52   simp
53   _ = x := GroupAction.ga_one x }

```

Then show the set is the carrier of a subgroup:

```

54   /-- The stabilizer set is the carrier of a subgroup of `G`. -/
55   theorem stabilizer_set_is_subgroup (x : X) :
56   ∃ H : Subgroup G, (H : Set G) = stabilizerSet (G := G) (X := X) x := by
57   refine ⟨stabilizer (G := G) (X := X) x, rfl⟩

```

7. What Lean Guarantees

Formalizing these constructions in Lean provides guarantees that informal proofs cannot match. When Lean accepts a definition or theorem, it has verified:

- **Types are correct.** The function $\text{phi} : G \rightarrow \text{Equiv.Perm } X$ has the claimed type. Lean checks that $\text{sigmaPerm } g$ is actually a permutation (a bijection), not just a function. If we mistakenly defined a non-bijective map, Lean would reject it.
- **No hidden assumptions.** Every lemma explicitly lists its hypotheses. If a proof uses associativity, the code must invoke `mul_assoc` or the `ga_mul` axiom. There are no “clearly” or “it follows that” steps that skip justification.
- **Axioms match definitions.** The `GroupAction` class requires `ga_mul` and `ga_one`. If we omitted `ga_one`, the proof of `phi_one` would fail because Lean would have no way to show $1 \cdot x = x$.
- **Subgroup structure is verified.** When we package the stabilizer as a `Subgroup`, Lean checks that we provided proofs of closure under multiplication, inverses, and identity. The type system prevents us from claiming “the stabilizer is a subgroup” without proving it.

These checks prevent common errors: mistaken claims about injectivity, forgotten hypotheses, and gaps in subgroup proofs. An external examiner can trust that the Lean code genuinely establishes the mathematical claims, not just that it compiles.

8. Reflection

Writing the proofs in Lean forced me to make every step explicit. In particular:

- I had to specify exactly where associativity of the action is used.
- I had to show explicitly that $\sigma_{g^{-1}}$ is an inverse.
- I had to unfold the definition of permutation multiplication.

The resulting code is not shorter than a textbook proof, but it is more precise. An external examiner can read the surrounding explanations and then see that each Lean line matches a specific mathematical claim.

I estimate the formalisation took over fifteen hours, mostly due to understanding Lean's typeclass resolution and the mechanics of equivalences and permutations in mathlib. The final development is compact but captures the standard argument for the permutation representation and the stabilizer subgroup.

8.1. Specific challenges encountered. Several technical hurdles emerged during the formalization:

- **Typeclass resolution:** Ensuring Lean could automatically find the `GroupAction` instance in complex contexts required careful use of explicit type annotations (e.g., `(G := G)`).
- **Equivalence mechanics:** Constructing `sigmaPerm` as an `Equiv.Perm X` required providing both `left_inv` and `right_inv` proofs, which meant carefully applying the `ga_mul` axiom in both directions.
- **Coercions:** Working with subgroups required understanding how Lean coerces elements of type `H` (a subgroup) to type `G` (the ambient group) using `(h : G)`.
- **Function extensionality:** Proving equality of permutations required `Equiv.ext`, and equality of vector-valued functions required `ext i`, which were not immediately obvious.

8.2. Potential extensions. This development could be extended in several directions:

- **Orbit-stabilizer theorem:** Formalize the relationship $|G| = |\text{Orb}(x)| \cdot |G_x|$ for finite groups, which requires defining orbits and proving Lagrange's theorem.
- **Burnside's lemma:** Count orbits using the formula involving fixed points.
- **Actions on cosets:** Show that a group acts on the left cosets of a subgroup, yielding the standard permutation representation.
- **Cayley's theorem:** Specialize the permutation representation to the case where G acts on itself, proving every group embeds in a symmetric group.

9. AI Assistance Statement

I used AI assistance to help draft Lean proofs and improve code readability. All AI-generated content was reviewed and verified by me to ensure mathematical accuracy.

10. References

John B. Fraleigh, Victor J. Katz, *A First Course in Abstract Algebra*, Addison–Wesley, 2003, Section 16 (Group Actions).